HW1: Variational Autoencoder (VAE)

Shubhan Mital
University of California San Diego
smital@ucsd.edu

Abstract

I train a convolutional variational autoencoder (VAE) in PyTorch on the Kaggle Chest X-Ray Pneumonia dataset. I report the hyperparameters, training curves, Inception Score (IS) and Fréchet Inception Distance (FID) of generated images, and samples. The final model reaches an FID of 211.03.

1 Method

Model. A VAE has a latent $z \sim \mathcal{N}(0, I)$, a decoder $p_\theta(x \mid z)$ and an approximate posterior $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x)))$. I minimize the negative ELBO

$$\mathcal{L}(x) = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] + \beta\,\mathrm{KL}\big(q_\phi(z \mid x)\,\|\,\mathcal{N}(0, I)\big),$$

with the reparameterization trick $z = \mu + \sigma \odot \epsilon$. The reconstruction term is the binary cross-entropy summed over pixels (images in $[0, 1]$), and $\mathcal{L}$ is averaged over the batch.

Architecture. Images are converted to grayscale and resized to 128×128. The encoder has five stride-2 4×4 convolutions (32–64–128–256–256 channels, BatchNorm, LeakyReLU) and two linear heads for μ and $\log \sigma^2$ (latent size 128). The decoder mirrors it with a linear layer, five transposed convolutions (BatchNorm, ReLU) and a sigmoid output; the model has 5.1M parameters (Figure 1). I train with Adam and cosine learning-rate decay.

Evaluation. IS and FID use Inception-v3 with ImageNet weights (grayscale replicated to 3 channels, resized to 299×299). FID compares 5000 generated images with 5000 real training images using 2048-d pooled features; IS uses 10 splits.

2 Experiments

Data and settings. I train on the `train` split (both classes pooled, labels unused) and monitor held-out loss on `test`. Table 1 gives the final settings and results (seed 0). They were chosen by a small sweep over latent size, β, learning rate, batch size and training length (36 runs, one seed each, ranked by FID).

Table 1: Hyperparameters and results of the final model.

Hyperparameter	Value	Metric	Value
Latent dim.	128	FID (↓)	211.03
Batch size	32	IS, generated (↑)	2.26 ± 0.04
Epochs	600	IS, real images	2.26 ± 0.06
Learning rate	0.001	Final train loss	9368.1
β (KL weight)	1	Final test loss	9458.9

Preprint.

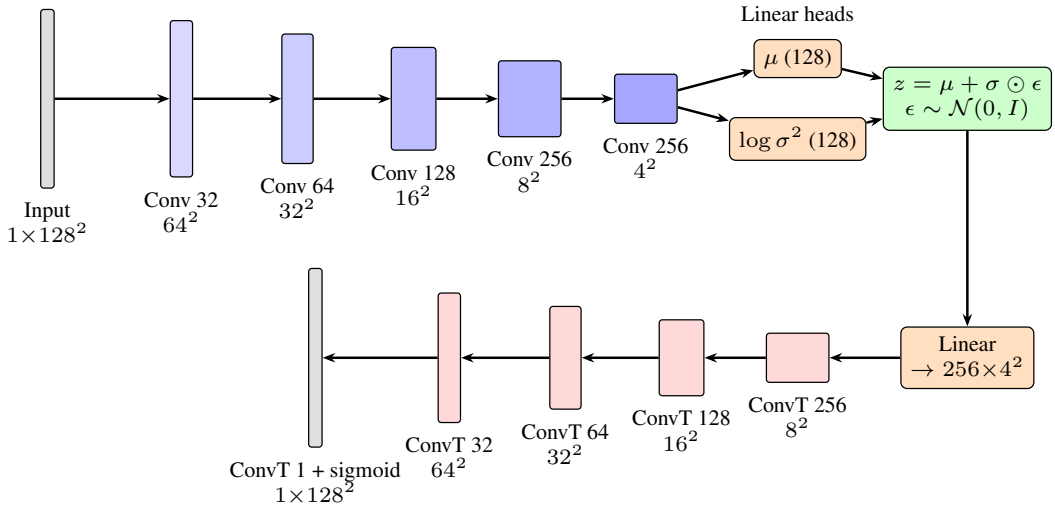


Figure 1: Architecture. Encoder: Conv 4×4, stride 2, BatchNorm, LeakyReLU. Decoder: ConvT 4×4, stride 2, BatchNorm, ReLU. A latent z is sampled with the reparameterization trick and decoded back to an image. Box height is proportional to spatial size and width to the number of channels.

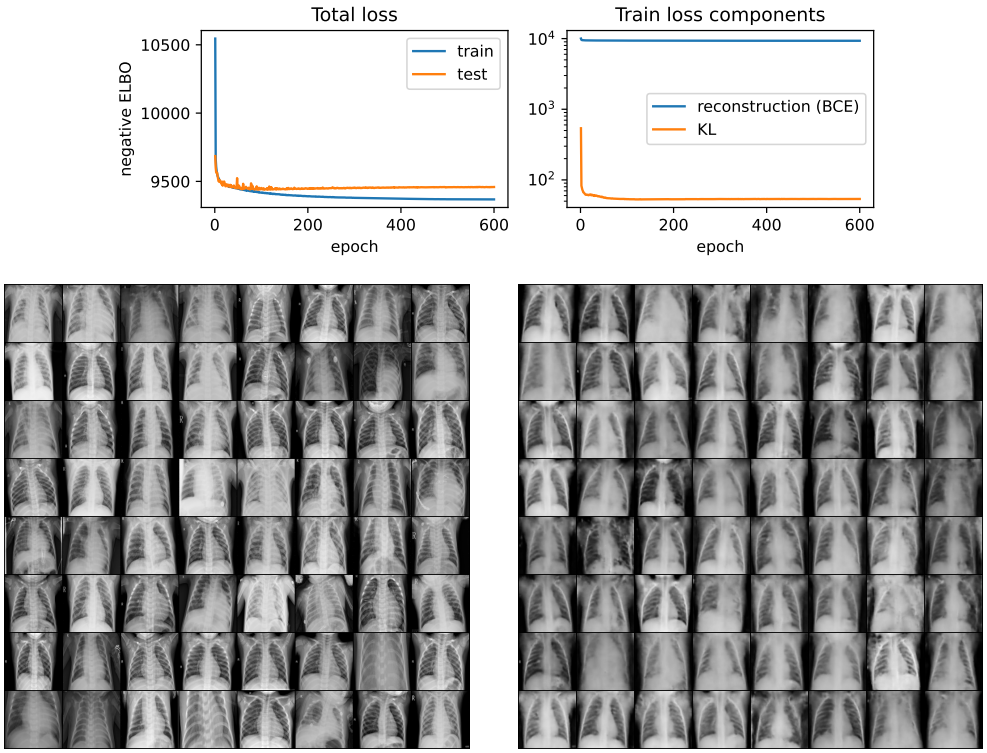


Figure 2: Top: training curves (log scale on the right). Bottom: real (left) and generated (right) images.

Training curves and samples. Figure 2 shows the results. The KL term settles near 54 nats and reconstruction dominates the loss. The test loss flattens after about 100 epochs and then rises slightly (train/test gap about 90 nats), while FID keeps improving.

3 Discussion

The samples show plausible thorax outlines, ribs and lungs but are smooth and lack fine texture, as is typical for a VAE with a pixel-wise likelihood. Held-out loss stopped improving after about 100 epochs, yet FID kept falling, so FID captures something the ELBO does not.

In the sweep, training length mattered most: with β=1, FID went from 244 at 60 epochs to 230 at 300 and 211 at 600. Smaller batches and a higher learning rate also helped (FID 268 for my first run at batch 64 and lr 5×10^{-4}, versus about 245 at batch 32 and lr 10^{-3}, both 60 epochs). Larger β improved FID at 60 epochs (298 for β=0.25, 268 for β=1, 256 for β=8) but lowered IS, and β=1 was best with long training. Latent size mattered little between 64 and 256. Each sweep entry is a single seed, so differences of a few FID points are within noise; re-training the best run gave 211.0 versus 210.5.

IS is equal for real and generated images (2.26 ± 0.06 vs. 2.26 ± 0.04) and low overall because Inception-v3 is trained on natural images, so FID is the more meaningful metric here.

Code. The code (`train_vae.py`) is available at `https://github.com/Shubhanflash22/ECE-285-Deep-Generative-Models/blob/main/Homework/HW1/train_vae.py`.